

Student-to-Tutor Scheduling and Availability Management System

WEB HOSTING & DOMAIN NAMING DOCUMENT

Detailed Final Version

Project Type	Individual project
Student	Nicolas Imhof
Student ID	700864476
Email	nimhof@oldwestbury.edu
Submitted To	Dr. Renu Balyan
Final Website URL	https://student-tutor-scheduling-system.pages.dev

Disclaimer: This document is prepared for course submission and system evaluation only. The system is an academic project and not intended for production academic advising or institutional scheduling without further review, security hardening, and formal testing.

TABLE OF CONTENTS

1. Introduction
2. Final Website URL and Domain Naming
3. Hosting and Cloud Services Used
4. Complete Download and Account Preparation
 - 4.1 Trae AI IDE
 - 4.2 Node.js/npm
 - 4.3 Git and GitHub
 - 4.4 Supabase
 - 4.5 Cloudflare Pages
5. GitHub Repository Upload Process
6. Cloudflare Pages Deployment Process
7. Supabase Database Hosting Process
8. Why This Deployment Method Was Chosen
9. Maintenance and Future Hosting Improvements
10. Troubleshooting
- Appendix A: References

1. Introduction

This document explains where the Student-to-Tutor Scheduling and Availability Management System is hosted, how the final URL/domain name was created, what services were used, and the step-by-step process needed to deploy the website. The project is hosted as a static website using Cloudflare Pages and connects to Supabase for authentication and database storage.

The deployment workflow is intentionally simple: edit and test the code in Trae AI, push the project to GitHub, connect the GitHub repository to Cloudflare Pages, and let Cloudflare publish the website to a free pages.dev URL. Supabase remains separate as the hosted database and authentication provider.

2. Final Website URL and Domain Naming

The final website URL is `https://student-tutor-scheduling-system.pages.dev`.

The domain name was not purchased from a paid domain registrar. It is a free Cloudflare Pages subdomain. Cloudflare Pages automatically provides a pages.dev URL based on the project name. In this case, the project name was set to student-tutor-scheduling-system, so the final address became student-tutor-scheduling-system.pages.dev.

Domain Detail	Value
Final URL	<code>https://student-tutor-scheduling-system.pages.dev</code>
Domain type	Free Cloudflare Pages subdomain
Paid domain purchased?	No
Custom DNS configured?	No
Hosting provider	Cloudflare Pages
Database provider	Supabase
Repository provider	GitHub

3. Hosting and Cloud Services Used

Service	Role in Project	Free or Paid
Cloudflare Pages	Hosts the static website files and provides the pages.dev URL.	Free tier
GitHub	Stores the repository that Cloudflare Pages deploys from.	Free account/repository
Supabase	Hosts PostgreSQL database, authentication, SQL functions, and seed data.	Free tier for academic prototype
Trae AI IDE	Coding editor and AI-assisted development environment.	Used as coding software; availability depends on Trae account/tool access
Node.js/npm	Local development tooling and package installation.	Free

4. Complete Download and Account Preparation

4.1 Trae AI IDE

Trae AI was used as the main coding software. It was used like VS Code would be used: open the project folder, edit files, run terminal commands, and review/debug code. The AI within Trae also helped generate database seed data, fix more complicated code errors, and streamline the GitHub repository upload process.

1. Go to <https://www.trae.ai/download>.
2. Choose TRAE IDE, not just TRAE SOLO, because the IDE is the editor used for code files.
3. Select Windows (x64) if using Windows 10 or 11.
4. Run the downloaded installer and accept the normal installation prompts.
5. Open Trae AI after installation.
6. Use File > Open Folder to open the extracted project folder.
7. Use Terminal > New Terminal to run npm and git commands from inside the project folder.

4.2 Node.js/npm

Node.js and npm are required for local package installation and local execution. npm installs @supabase/supabase-js and http-server from package.json.

8. Go to the official Node.js website and download the LTS installer.
9. Run the installer and leave npm selected.
10. Restart Trae AI after installation.
11. Open Trae terminal and verify with node -v and npm -v.
12. Inside the project folder, run npm install.
13. Run npm start to start the local http-server.

```
node -v
npm -v
npm install
npm start
```

4.3 Git and GitHub

GitHub was used to store the project repository. Git is recommended for uploading from the command line, although GitHub web upload or GitHub Desktop could also be used. Trae AI helped streamline the process of preparing the files and uploading them to the GitHub repository.

14. Create or log into a GitHub account.
15. Install Git from <https://git-scm.com/downloads> if using terminal commands.
16. Create a new GitHub repository. (Do not add an extra README if pushing an existing project that already has one, unless you know how to resolve merge conflicts)
17. Open the project folder in Trae terminal.
18. Run git init if the folder is not already a Git repository.
19. Run git add . to stage the project files.
20. Run git commit -m "Initial project upload".
21. Add the GitHub remote using git remote add origin <repository-url>.
22. Run git branch -M main.
23. Run git push -u origin main.

```
git init
git add .
git commit -m "Initial project upload"
git remote add origin <repository-url>
git branch -M main
git push -u origin main
```

4.4 Supabase

Supabase hosts the database and authentication. It is not installed like a desktop program. Instead, the project uses the Supabase website/dashboard and the Supabase JavaScript client.

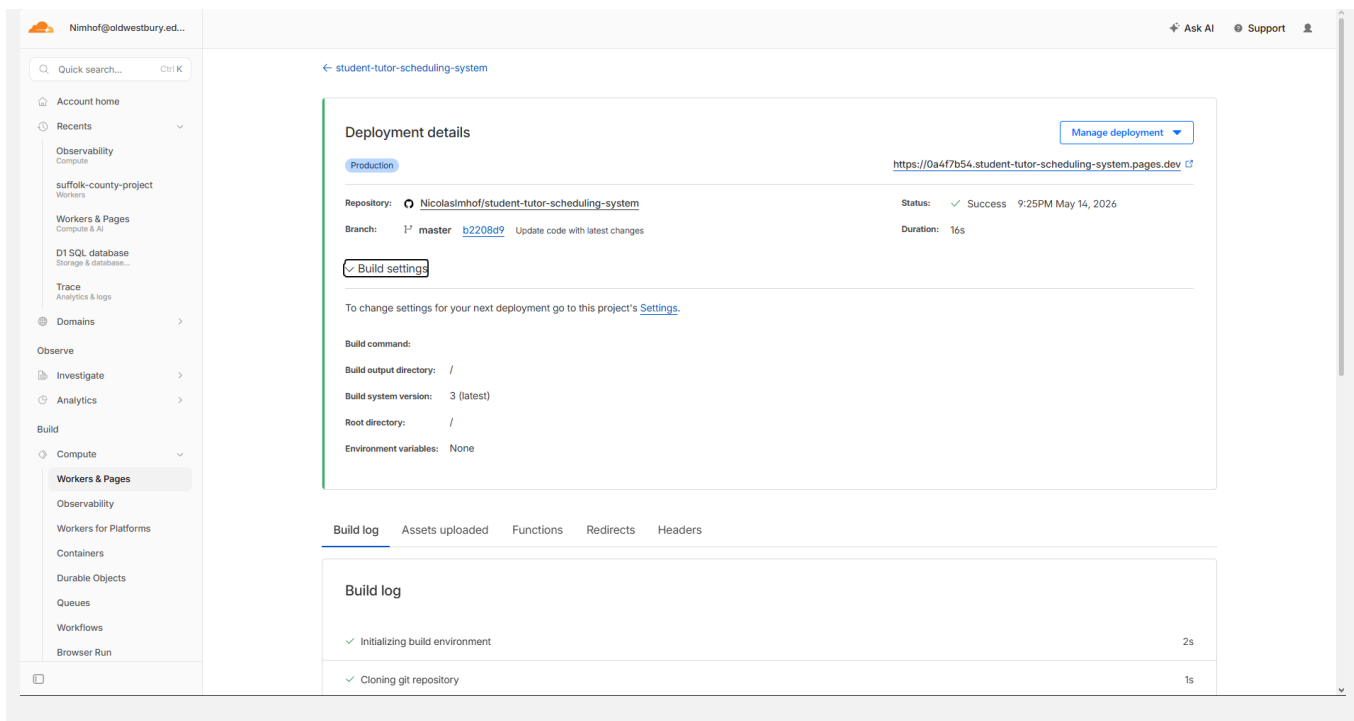
24. Create or log into a Supabase account.
25. Create a new Supabase project.

26. Open Project Settings > API and copy the Project URL and anon public key.
27. If using a fresh project, update script.js with the new Supabase URL and anon key.
28. Open SQL Editor in Supabase.
29. Run the migrations and seed.sql files included in the project.
30. Confirm the users, courses, appointments_enhanced, working_hours, reviews, time_off_requests, and system_events tables contain data.
31. Confirm Supabase Authentication contains matching demo users if login testing is needed.

4.5 Cloudflare Pages

Cloudflare Pages hosts the deployed static site. No paid domain was purchased. The project uses the free pages.dev subdomain.

32. Create or log into a Cloudflare account.
33. Go to Workers & Pages in the Cloudflare dashboard.
34. Choose Create application, then Pages, then Connect to Git.
35. Authorize Cloudflare to access the GitHub repository.
36. Select the student-tutor-scheduling-system repository.
37. Set the production branch to main.
38. Because this is a static HTML/CSS/JavaScript project, leave the build command blank unless Cloudflare requires a value.
39. Set the output directory/root to the repository root if the HTML files are in the root folder.
40. Click Save and Deploy.
41. After deployment, open the provided pages.dev URL and test the website.



5. GitHub Repository Upload Process

The GitHub upload process is important because Cloudflare Pages deploys from GitHub. If GitHub does not contain the correct files, Cloudflare cannot deploy the correct website. The files that must be present include index.html,

register.html, script.js, calendar.js, style.css, calendar.css, package.json, package-lock.json, migrations, supabase/seed.sql, and README.md.

Files that should not be uploaded include node_modules, .env, local logs, temporary files, and secret keys. The .gitignore file is used to prevent these from being committed.

- Safe to upload: source code, CSS, HTML, public Supabase anon key, SQL migrations, README, package files.
- Not safe to upload: Supabase service role key, .env files, passwords used for real users, node_modules, local cache files.

6. Cloudflare Pages Deployment Process

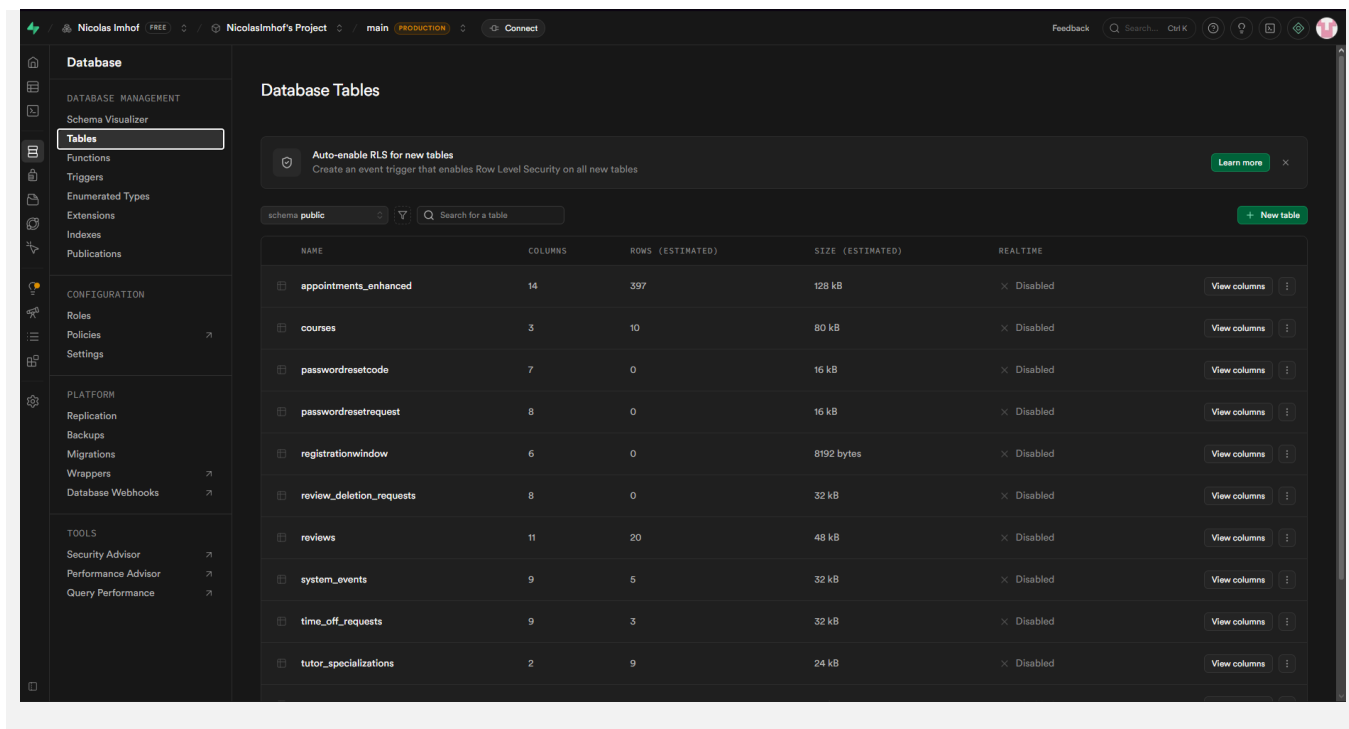
Cloudflare Pages was selected because the project front end is static and does not require a separate Node.js server in production. The browser can load the static files from Cloudflare, and Supabase handles the database and authentication work separately.

42. Push the final code to GitHub.
43. Open Cloudflare dashboard > Workers & Pages.
44. Create a Pages project connected to GitHub.
45. Select the repository and main branch.
46. Use no framework preset or choose None/Static if available.
47. Leave build command blank for a plain static site.
48. Use repository root as the output directory if index.html is at the root.
49. Deploy and wait for Cloudflare to finish the build.
50. Open the deployment URL and test login, registration, dashboard, booking, calendar, reviews, and admin features.

7. Supabase Database Hosting Process

Supabase is the hosted back-end. The deployed Cloudflare site depends on Supabase being available because login, dashboard data, tutor lists, bookings, reviews, and user approvals all require Supabase requests.

- Supabase Auth stores login accounts and sessions.
- public.users stores profile data and roles.
- SQL tables store appointments, reviews, courses, time off, working hours, and system events.
- RPC functions keep booking and calendar logic consistent.
- Seed files provide realistic test data for demonstration.



8. Why This Deployment Method Was Chosen

The project was coded using methods that match what I know of web development knowledge. HTML, CSS, JavaScript, and SQL are direct and readable. A static hosting provider is enough because the front-end files do not require server-side rendering. Supabase handles authentication and PostgreSQL data, so a separate Express server was not necessary for the deployed version.

9. Maintenance and Future Hosting Improvements

Future Item	Explanation
Custom domain	Purchase a domain and connect it to Cloudflare DNS if the project becomes a real public service.
Environment variables	Move all private keys into secure provider settings and remove any key-like strings from source files.
CI testing	Add automated tests before Cloudflare deploys each new version.
Staging branch	Use a preview branch for testing before pushing to production main.
Monitoring	Add Cloudflare Web Analytics and Supabase logs review to monitor failures.

10. Troubleshooting

Issue	Likely Cause	Fix
Cloudflare deploy shows old version	GitHub main branch was not updated or deployment cache is still finishing.	Confirm git push succeeded, then redeploy or wait for Pages build to complete.
404 at deployed URL	Wrong output directory or missing index.html at root.	Set output/root directory to the folder containing index.html.

Login fails on deployed site	Supabase URL/key mismatch, user not in Auth, or user pending approval.	Check script.js URL/key, Supabase Auth, and users.approval_status.
npm install fails	Node/npm not installed correctly or terminal path is wrong.	Verify node -v/npm -v and cd into the project root before running npm install.
Git push fails	Remote not configured, branch mismatch, or login/auth problem.	Run git remote -v, git branch -M main, and sign in to GitHub or use a token if prompted.

Appendix A: References

Reference	Location
Trae AI Download	https://www.trae.ai/download
Trae AI Quickstart	https://docs.trae.ai/ide/set-up-trae?_lang=en
npm Docs - Downloading and installing Node.js and npm	https://docs.npmjs.com/downloading-and-installing-node-js-and-npm/
Supabase JavaScript Installing	https://supabase.com/docs/reference/javascript/installing
Supabase JavaScript Introduction	https://supabase.com/docs/reference/javascript/introduction
GitHub Docs - Adding locally hosted code to GitHub	https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github
Cloudflare Pages - Git integration	https://developers.cloudflare.com/pages/get-started/git-integration/
Cloudflare Pages - Direct Upload	https://developers.cloudflare.com/pages/get-started/direct-upload/